



Research Intake 2026

Day 12

Introduction to Text Classification

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Table of Contents

No	Section	Subsections
1	Text Classification — Problem Formulation	1.1 Taxonomy of Classification Settings 1.2 Formal Problem Definition 1.3 Benchmarks & Datasets
2	The Full Classification Pipeline	2.1 Seven-Stage Architecture 2.2 Data Annotation 2.3 Distribution Shift & Monitoring
3	Feature Engineering for Text	3.1 Bag-of-Words 3.2 TF-IDF 3.3 N-grams 3.4 Word Embeddings 3.5 Contextual Embeddings
4	Naive Bayes Classifier	4.1 Probabilistic Theory 4.2 Laplace & Lidstone Smoothing 4.3 Complement NB 4.4 Practical Guide
5	Logistic Regression for NLP	5.1 Log-Odds & Sigmoid 5.2 Softmax for Multi-class 5.3 Regularisation 5.4 Optimisation 5.5 Interpretability
6	Evaluation Metrics	6.1 Confusion Matrix 6.2 P/R/F1 6.3 Averaging Methods 6.4 ROC-AUC & PR-AUC 6.5 Calibration
7	Cross-Validation & Model Selection	7.1 K-Fold Variants 7.2 Grid vs. Random vs. Bayesian Search 7.3 Nested CV
8	Class Imbalance	8.1 Diagnosis 8.2 SMOTE & Variants 8.3 Class Weights 8.4 Threshold Moving 8.5 Ensemble Methods
9	Deep Learning for Text Classification	9.1 CNN Text Classifiers 9.2 LSTM/BiLSTM 9.3 BERT Fine-Tuning 9.4 Model Selection Guide
10	Research Frontiers	10.1 Zero-Shot 10.2 Few-Shot & SetFit 10.3 Prompt Engineering 10.4 Open Problems
A	Python Implementation Appendix	Sklearn Pipeline Evaluation Dashboard BERT Fine-Tune Skeleton

Text classification is the act of assigning one or more discrete labels to a piece of text — and it is simultaneously the simplest and most commercially significant task in Natural Language Processing. From the spam filter protecting your inbox to the content moderation system reviewing social media posts at petabyte scale, text classification sits at the foundation of modern AI infrastructure. The challenge is not in defining the problem, but in navigating the enormous space of design decisions that determine production performance: Which feature representation best captures the signal in your domain? Which evaluation metric aligns with the actual cost of errors in your application? How do you handle the near-universal reality of imbalanced class distributions? When is a 100-parameter logistic regression better than a 110-million-parameter transformer?

This document provides a principled, research-grounded answer to each of these questions. It is structured in ten sections, progressing from foundational problem formulation through classical models, rigorous evaluation, class imbalance handling, deep learning approaches, and culminating in the current research frontier of zero-shot and few-shot classification. A practical Python appendix closes the document with implementation-ready code skeletons. The central thesis: a practitioner who understands Naive Bayes, Logistic Regression, and their evaluation deeply will outperform one who jumps straight to transformers without that foundation. Classical models train in seconds, are trivially interpretable, and frequently achieve 90% of the performance of BERT at 1% of the cost.

1

Text Classification: Problem Formulation

Taxonomy, formal definition, benchmarks, and real-world scope

1.1 What is Text Classification?

Text classification is a supervised learning task that maps an input document d (a string of arbitrary length) to one or more labels from a predefined label set $L = \{l_1, l_2, \dots, l_k\}$. The function $f: D \rightarrow L$ is learned from a labelled training corpus $\{(d_1, y_1), \dots, (d_n, y_n)\}$ where each $y_i \in L$.

The 'predefined' constraint distinguishes text classification from open-domain tasks such as text generation or information extraction. In classification, the label space is fixed at training time. This constraint makes the problem tractable — but it also means that any label not in L is invisible to the classifier, regardless of how frequently it appears in production data. Managing label drift over time is one of the most under-discussed challenges in production NLP systems.

Text Classification (Formal)

Given a document $d \in D$ and a predefined label set $L = \{l_1, \dots, l_k\}$, learn a function $f: D \rightarrow L$ (or $f: D \rightarrow 2^L$ for multi-label) that minimises expected loss $E[\text{Loss}(f(d), y)]$ over the true data distribution $P(d, y)$.

1.2 Taxonomy of Classification Settings

The problem formulation varies significantly depending on the structure of the output space. Each variant requires different loss functions, evaluation protocols, and architectural choices.

Setting	Output Space	Loss Function	Evaluation
Binary	$y \in \{0, 1\}$	Binary cross-entropy	F1, ROC-AUC, PR-AUC
Multi-class (OvR)	$y \in \{1, \dots, K\}$	Categorical cross-entropy (softmax)	Macro/weighted F1, confusion matrix
Multi-label	$y \subseteq \{1, \dots, K\}$ (subset)	Binary cross-entropy per label (sigmoid)	Micro F1, Hamming loss, label ranking AP
Hierarchical	y is a path in a taxonomy tree	Hierarchical cross-entropy or flat softmax	Hierarchical F1, depth-weighted accuracy
Ordinal	$y \in \{1, \dots, K\}$ with ordering	Ordinal regression loss (QWK)	Quadratic Weighted Kappa (QWK), MAE

Ordinal classification is frequently overlooked but common in practice: star ratings (1–5), severity scores (mild/moderate/severe), and urgency levels all have inherent ordinal structure. Treating them as nominal multi-class discards valuable information and typically produces worse calibration at the extremes.

1.3 Formal Problem Definition and the i.i.d. Assumption

Standard supervised learning assumes that training examples $(d_1, y_1), \dots, (d_n, y_n)$ are drawn independently and identically distributed (i.i.d.) from the true data distribution $P(d, y)$. This assumption is almost always violated in practice:

- Temporal correlation: Documents written close in time share vocabulary, topic, and style

- Source correlation: All documents from the same publication share editorial style
- Annotation bias: Human annotators have systematic disagreements that vary by annotator

Understanding these violations is critical for honest evaluation. A model achieving 95% accuracy on a test set drawn from the same news outlet as the training set may achieve only 78% on documents from a different publication — a 17-point gap caused entirely by violating the i.i.d. assumption.

Key Takeaway

Always evaluate your classifier on data that matches the distribution it will encounter in production. A geographically diverse, temporally diverse test set is far more informative than a large test set drawn from the same source as training data.

1.4 Common Benchmarks and Datasets

Benchmarks provide standardised evaluation conditions that enable fair comparison across methods. The following datasets are the most widely used in the NLP classification literature:

Dataset	Task	Scale	Access	Key Challenge
IMDb Reviews	Binary sentiment	50K reviews	Maas et al. (2011)	Long reviews; sarcasm; domain-specific vocab
AG News	4-class topic	120K articles	Zhang et al. (2015)	Topic boundary ambiguity
20 Newsgroups	20-class topic	18K posts	scikit-learn built-in	Overlapping categories; header leakage
SST-2	Binary fine-grained sentiment	11K sentences	GLUE benchmark	Short phrases; implicit sentiment
TREC-6	6-class question type	5.5K questions	TREC website	Short texts; ambiguous intent
Reuters-21578	Multi-label news topics	~10K articles	NLTK corpus	Severe class imbalance; multi-label
Yelp Polarity	Binary sentiment	560K reviews	Zhang et al. (2015)	Large scale; colloquial language
DBpedia	14-class ontology	630K articles	Zhang et al. (2015)	Wikipedia-style; clean text

Warning: 20 Newsgroups Header Leakage

The 20 Newsgroups dataset contains newsgroup headers (e.g. 'From: rec.sport.hockey@...') that trivially identify the class. Classifiers that use the raw data — rather than stripping headers — report inflated accuracy of >99% that does not reflect genuine content understanding. Always use the `subset='all'`, `remove headers=True`, `remove footers=True`, `remove quotes=True` settings in `sklearn`.

2

The Full Classification Pipeline

From raw data to monitored production system

A text classifier is not a model — it is a system. The model is the simplest component. The hardest problems lie in data collection, annotation quality, feature engineering decisions, evaluation design, and production monitoring. This section traces the full pipeline with the depth required for practical implementation.

2.1 Stage 1 — Data Collection and Scope Definition

The quality of your classifier is bounded by the quality of your data. Before writing any code, three questions must be answered with precision:

1. What exactly does each label mean? Write explicit label definitions with inclusion/exclusion criteria and at least 10 canonical examples per class.
2. What is the source distribution of production data? Your training data must match it. If you collect data from formal news articles but deploy on social media posts, you have a distribution mismatch problem that no model can fix.
3. Who will annotate, and how will you measure their agreement? The inter-annotator agreement (Cohen's Kappa) should be computed on a minimum 10% overlap sample before full annotation begins.

Annotation Quality Metric	Formula	Interpretation	Target
Cohen's Kappa (κ)	$(P_o - P_e) / (1 - P_e)$	Agreement beyond chance	$\kappa > 0.70$ for most tasks; $\kappa > 0.80$ for medical
Fleiss' Kappa	Multi-annotator generalisation of Cohen's	Used when >2 annotators per item	Same thresholds as Cohen's
Krippendorff's Alpha	Handles missing data; generalises to ordinal	Most robust for irregular annotation	$\alpha > 0.667$ minimum

2.2 Stage 2 — Text Preprocessing

Preprocessing converts raw text into a canonical form that reduces vocabulary sparsity and emphasises semantically relevant content. The right preprocessing choices are domain-dependent.

Technique	Description	Use When	Avoid When
Lowercasing	Convert all text to lowercase	Case is not semantically meaningful	Named entity recognition; case carries meaning
Tokenisation	Split text into tokens (words, subwords, characters)	Always required as a first step	(Always apply)
Stopword removal	Remove high-frequency function words (the, is, at)	Bag-of-words models with large vocab	Transformer models (handle stopwords natively)
Stemming	Reduce words to root form via heuristics (running→run)	Resource-constrained settings	Precision applications; introduces errors

Technique	Description	Use When	Avoid When
Lemmatisation	Reduce to dictionary base form using morphology	Higher quality than stemming	Speed-critical pipelines
Punctuation removal	Strip punctuation marks	Simple topic classification	Sentiment (! and ? carry sentiment)
URL/mention normalisation	Replace URLs with <URL>, @names with <USER>	Social media data	Formal text corpora

Key Takeaway

For transformer-based models (BERT, RoBERTa), minimal preprocessing is best: do not remove stopwords, do not stem or lemmatise, and preserve punctuation. These models were pre-trained on raw text and handle linguistic variation internally.

2.3 Stage 7 — Deployment and Distribution Shift Monitoring

The most dangerous assumption in production ML is that the data distribution remains stable after deployment. In text classification, distribution shift manifests as:

- Concept drift: The meaning or boundaries of labels change over time (e.g. 'AI news' in 2018 vs. 2024)
- Covariate shift: The vocabulary and style of incoming documents shifts (seasonal topics; major events)
- Label shift: The prior probability of classes changes (e.g. more complaints during product outages)

Production monitoring should track: (1) prediction confidence distribution — a drop in mean confidence signals distribution shift; (2) label marginal distribution — flag when predicted class proportions diverge $>2\sigma$ from training priors; (3) held-out human evaluation on a weekly sample of 100 documents.

Use Case: Twitter Content Moderation

During major world events (elections, disasters, viral controversies), the distribution of hate speech, misinformation, and sensitive content on social platforms shifts dramatically within hours. A classifier trained on a static corpus may see its precision drop from 85% to 62% within 24 hours. Twitter's production system includes real-time feedback loops: human reviewer decisions are incorporated into retraining within 6–12 hours during high-volume events.

3

Feature Engineering for Text

Bag-of-Words, TF-IDF, N-grams, embeddings, and contextual representations

Feature engineering transforms raw text strings into numerical vectors that machine learning models can process. The choice of representation is one of the highest-leverage decisions in the entire classification pipeline — often more impactful than the choice of model architecture.

3.1 Bag-of-Words (BoW)

The Bag-of-Words model represents each document as a vector of word counts, ignoring word order and grammar. Given a vocabulary V of size $|V|$, each document becomes a $|V|$ -dimensional vector where entry i is the count of word v_i in the document.

Bag-of-Words Representation

$\mathbf{d} = [\text{count}(w_1, \mathbf{d}), \text{count}(w_2, \mathbf{d}), \dots, \text{count}(w_n, \mathbf{d})]$ where $|\mathbf{d}| = |V|$

V is typically 10,000–100,000 terms after vocabulary pruning. The resulting matrix is extremely sparse: a document containing 200 unique words has 200 non-zero entries out of 50,000 — a sparsity of 99.6%.

BoW vectors are the fundamental representation for classical classifiers. Their simplicity belies their effectiveness: on many topic classification tasks, a Naive Bayes model with BoW features approaches the performance of more complex representations.

Vocabulary Pruning Strategy

Raw text corpora contain vast numbers of hapax legomena (words appearing only once) that consume memory without providing discriminative signal. Standard practice: (1) remove terms appearing in fewer than $\text{min_df}=2$ –5 documents; (2) remove terms appearing in more than $\text{max_df}=0.95$ of documents (these are not discriminative); (3) cap vocabulary at $\text{max_features}=50,000$ ordered by term frequency. This reduces vocabulary from 200K+ to ~30K with minimal performance impact.

3.2 TF-IDF Weighting

Raw word counts give equal importance to high-frequency words that appear in nearly every document (which carry no discriminative information) and rare words that are highly specific to certain classes. TF-IDF corrects this by down-weighting ubiquitous terms and up-weighting rare, discriminative ones.

TF-IDF Formula

$\text{tfidf}(w, \mathbf{d}, \mathbf{D}) = \text{tf}(w, \mathbf{d}) \times \text{idf}(w, \mathbf{D}) = \log(1 + \text{count}(w, \mathbf{d})) \times \log((N+1) / (\text{df}(w)+1)) + 1$

N = total documents; $\text{df}(w)$ = documents containing w ; the $+1$ terms implement sublinear scaling and prevent division-by-zero. `sklearn's TfidfVectorizer` applies L2 normalisation per document row by default, making cosine similarity equivalent to dot product.

Variant	Formula	Effect
Raw TF	$\text{count}(w,d)$	Proportional to raw frequency; large documents dominate
Log TF	$\log(1 + \text{count}(w,d))$	Sublinear scaling; reduces impact of very frequent words
Binary TF	1 if count > 0 else 0	Presence/absence only; useful for short texts
Standard IDF	$\log(N/\text{df}(w))$	Can produce -0 for universal terms; less stable
Smooth IDF (default)	$\log((N+1)/(\text{df}(w)+1)) + 1$	Adds 1 to avoid division by zero; sklearn default

3.3 N-gram Features

Unigram (single-word) BoW loses all sequential information. N-gram models extend BoW to sequences of N consecutive tokens, capturing short-range dependencies such as negation ('not good'), compound concepts ('machine learning'), and idiomatic phrases. Bigrams (N=2) consistently improve text classification performance at low computational cost. Trigrams (N=3) add further signal for some tasks but dramatically increase vocabulary size. In practice, using `ngram_range=(1,2)` in sklearn's `TfidfVectorizer` is a reliable default that improves over unigrams without intractable sparsity.

N-gram Vocabulary Explosion Warning

A vocabulary of 50,000 unigrams expands to approximately 800,000 bigrams and 3.2 million trigrams in a typical news corpus. Without aggressive `min_df` pruning (`min_df ≥ 3` for bigrams), the resulting matrices exceed memory limits. Always set `min_df ≥ 3` when using bigrams and `min_df ≥ 10` for trigrams.

3.4 Word Embeddings

Sparse BoW vectors treat every word as orthogonal — 'cat' and 'kitten' are as dissimilar as 'cat' and 'democracy'. Word embeddings map words to dense vectors in a continuous space where semantic similarity corresponds to geometric proximity. Words with similar distributional contexts (appearing with similar neighbours) receive similar vectors.

Embedding Method	Dimensions	Trained On	Key Property
Word2Vec (CBOW)	50–300	Google News (100B words)	Syntactic: king - man + woman ≈ queen
Word2Vec (Skip-gram)	50–300	Same	Rare word representation; slightly better for analogies
GloVe	50–300	Common Crawl (840B tokens)	Co-occurrence statistics; stable for factual associations

Embedding Method	Dimensions	Trained On	Key Property
FastText	300	Wikipedia + Common Crawl	Subword-aware; handles OOV words; better for morphological languages
Sentence-BERT	768	MS MARCO, NLI datasets	Sentence-level; optimal for SetFit and semantic search

For document classification with static embeddings, the standard approach is to average all word vectors in the document: $d_vec = (1/|d|) \sum w_vec(w_i)$. This destroys word order but produces a fixed-size vector regardless of document length. Weighted averaging by TF-IDF scores improves performance: rare, informative words contribute more to the document representation.

3.5 Contextual Embeddings (BERT-family)

The fundamental limitation of static embeddings (Word2Vec, GloVe, FastText) is that each word has a single vector regardless of context: 'bank' in 'river bank' and 'bank account' receive identical representations. Contextual embeddings generate different representations for the same word depending on its surrounding context. BERT (Bidirectional Encoder Representations from Transformers) produces contextual token embeddings by running a deep transformer network over the full input sequence. The [CLS] token representation — the first token in every BERT input — aggregates information from the entire sequence and is commonly used as the document representation for classification tasks.

When to Use Contextual vs. Static Embeddings

Use TF-IDF or static embeddings when: training data < 500 examples per class; inference latency < 10ms is required; interpretability is a hard requirement; the task is simple topic classification without semantic nuance. Use BERT-family models when: semantic understanding matters (sentiment, intent, NLI); fine-tuning data is available (>200 examples per class); accuracy is the primary optimisation target and compute budget is available.

4.1 Probabilistic Foundations

Naive Bayes is a generative probabilistic classifier. Rather than learning a direct mapping from documents to labels (discriminative), it models the joint probability $P(\text{document}, \text{class})$ and applies Bayes' theorem to infer $P(\text{class} \mid \text{document})$. This distinction has profound implications for its behaviour under distribution shift and with limited data.

Bayes' Theorem Applied to Classification

$$P(c \mid d) = P(c) \times P(d \mid c) / P(d) \quad [P(d) \text{ is constant across classes, so ignored}]$$

We classify document d to class $c^ = \operatorname{argmax}_c P(c) \times P(d \mid c)$. $P(c)$ is the class prior; $P(d \mid c)$ is the class-conditional likelihood.*

The 'naive' conditional independence assumption states that given the class label c , all word features are statistically independent: $P(d \mid c) = \prod_i P(w_i \mid c)^{\text{count}_i}$. This assumption is demonstrably false in natural language — 'New' and 'York' are highly correlated — but the classification decision depends only on the relative rankings of $P(c \mid d)$ across classes, not its absolute value. Fortunately, the rankings are often preserved even when the probability estimates are poor.

Multinomial NB in Log Space (numerically stable)

$$\log P(c \mid d) \propto \log P(c) + \sum_i \text{count}(w_i, d) \times \log P(w_i \mid c)$$

Log space converts the product of many small probabilities (which would underflow to 0) into a sum of log probabilities. This is numerically stable even for documents with hundreds of tokens.

4.2 Laplace Smoothing and the Vocabulary Problem

When a word w_i appears in the test document but never appeared in any training document for class c , $P(w_i \mid c) = 0$. This single zero probability causes the entire product $P(d \mid c)$ to collapse to zero, making it impossible to assign d to class c regardless of all other evidence. Laplace smoothing prevents this:

Laplace (Add-1) Smoothing

$$P(w \mid c) = (\text{count}(w, c) + \alpha) / (\sum_v \text{count}(v, c) + \alpha \times |V|)$$

$\alpha=1$ is Laplace smoothing; $\alpha < 1$ is Lidstone smoothing (a hyperparameter tunable via cross-validation). $|V|$ is the full vocabulary size. With $\alpha=1$, even unobserved words receive a small non-zero probability proportional to $1/|V|$.

The optimal α depends on vocabulary size and training set size. For very large vocabularies ($|V| > 100,000$), $\alpha < 1$ is often better because $\alpha=1$ over-smooths rare words. Tune α on a validation set using values $\{0.01, 0.1, 0.5, 1.0, 2.0\}$.

4.3 Complement Naive Bayes (CNB)

Standard Multinomial NB estimates $P(w|c)$ from documents of class c . Complement NB (CNB), introduced by Rennie et al. (2003), instead estimates the probability of a word given all classes except c — the 'complement' set. This corrects for the poor probability estimates produced when training classes are imbalanced.

Complement NB Classification

$$c^* = \operatorname{argmin}_c \sum_i \text{count}(w_i, d) \times \log P(w_i | \bar{c})$$

We choose the class whose complement explains the document least well. CNB is the default variant in sklearn (ComplementNB) and typically outperforms MultinomialNB by 1–3% on imbalanced datasets.

4.4 Practical Guide: When to Choose Naive Bayes

Scenario	NB Verdict	Reason
< 1,000 training examples per class	Recommended	Low-data regime favours generative models
Real-time inference (< 1ms latency)	Recommended	Inference is a single dot product per class
Streaming data classification	Recommended	NB supports online (incremental) learning
Multi-label classification	Use with caution	Independence assumption breaks down across labels
Sentiment analysis with negation	Disfavoured	'not good' BoW loses the dependency
> 10K examples, complex semantics	Disfavoured	Logistic regression or transformers will outperform

5.1 Discriminative vs. Generative Models

Naive Bayes is generative: it models $P(x, y) = P(y)P(x|y)$ and classifies via Bayes' rule. Logistic Regression is discriminative: it directly models $P(y|x)$ without any assumptions about how x is generated. This distinction has practical consequences:

- Discriminative models require more data to converge but typically achieve better accuracy when enough data is available (Ng & Jordan, 2002)
- Generative models are more robust with very limited data and can naturally handle missing features
- Discriminative models are more sensitive to feature correlations but can exploit them once trained

Logistic Regression Decision Boundary

$$\log(p / (1-p)) = w^T x + b \Rightarrow p = \sigma(w^T x + b) \Rightarrow \hat{y} = 1 \text{ if } p \geq 0.5 \text{ else } 0$$

The decision boundary is the hyperplane $w^T x + b = 0$. In high-dimensional TF-IDF space ($|V| = 50,000$), this hyperplane separates two half-spaces of document feature vectors.

5.2 Softmax Generalisation for Multi-Class

For $K > 2$ classes, logistic regression generalises via the softmax function. There are K weight vectors $\{w_1, \dots, w_k\}$, one per class. Classification assigns the document to the class with the highest softmax score:

Softmax Multi-Class Logistic Regression

$$P(y=k | x) = \exp(w_k^T x + b_k) / \sum \exp(w^T x + b) \quad c^* = \operatorname{argmax}_k P(y=k|x)$$

The softmax ensures the K class probabilities sum to 1. The weight matrix W has dimensions $K \times |V|$; for $|V|=50,000$ and $K=20$, this is 1,000,000 parameters. L2 regularisation is critical to prevent overfitting.

5.3 L1/L2/ElasticNet Regularisation — Deep Dive

Regularisation adds a penalty term to the training objective that discourages large weights. This is essential for text classification where $|V| \gg N$ (vocabulary far exceeds training set size).

Regulariser	Penalty	Geometric Interpretation	Text Classification Use Case
L2 (Ridge, C in sklearn)	$\ w\ ^2 = \sum w_i^2$	Penalises large coefficients; keeps all features; elliptical constraint	Default for most tasks; handles correlated features
L1 (Lasso)	$\ w\ _1 = \sum w_i $	Promotes sparsity; drives many weights to exactly zero; diamond constraint	Interpretability; feature selection over 50K+ vocab

Regulariser	Penalty	Geometric Interpretation	Text Classification Use Case
ElasticNet	$r\ w\ + (1-r)\ w\ ^2$	Sparse like L1 but stable with correlated groups	Very high-dimensional vocab with correlated word groups

In sklearn's LogisticRegression, the regularisation strength is controlled by $C = 1/\lambda$ — smaller C means stronger regularisation. The default $C=1.0$ is often a good starting point, but C should always be tuned via cross-validation on the range $\{0.001, 0.01, 0.1, 1, 10, 100\}$.

5.4 Optimisation Algorithms

The log-likelihood objective of logistic regression is convex — there is a unique global minimum. Multiple optimisation algorithms can find it, with different speed-memory trade-offs:

Solver	Best For	Memory	Notes
lbfgs (default)	Small-medium datasets; L2 regularisation	High (stores Hessian approx.)	Fast convergence; not suitable for very large N
saga	Large sparse datasets; L1 or ElasticNet	Low	Stochastic gradient; good for NLP sparse features
liblinear	Binary classification; L1 regularisation	Medium	Coordinate descent; robust but doesn't scale to multi-class well
sag	Large dense datasets; L2 only	Low	Stochastic average gradient; fastest for dense data

5.5 Interpreting Learned Weights for Debugging

The weight vector of a trained logistic regression is directly interpretable: w_i represents the contribution of feature i (a word or n-gram) to the log-odds of the positive class. This interpretability is a first-class advantage over neural models.

Extracting Top Features per Class (Python/sklearn)

```
feature_names = vectorizer.get_feature_names_out()
weights = classifier.coef_[class_index] # shape: (|V|,)
top_positive = feature_names[np.argsort(weights)[-20:]] # top 20 for class
top_negative = feature_names[np.argsort(weights)[:20]] # bottom 20
# Produces interpretable lists like:
# Positive: ['excellent', 'outstanding', 'love', 'brilliant', ...]
# Negative: ['terrible', 'awful', 'waste', 'disappointing', ...]
```

Use Case: Healthcare NLP Compliance

A hospital system classifies clinical notes for billing code validation using L1-regularised LR. The model identifies 38 non-zero features. These are reviewed by coding specialists and embedded in a 'coding checklist' displayed to clinicians during note entry — reducing miscoding by 23% in a 6-month trial. A BERT-based model achieved 2% higher F1 but was rejected by the compliance team because it could not explain its decisions.

6.1 Why Accuracy Fails

Consider a dataset with 950 negative examples and 50 positive examples. A classifier that always predicts 'negative' achieves 95% accuracy. It has recall=0.0 on the positive class, which is the class that matters in most applications (fraud, disease, spam). Accuracy is a valid metric only when classes are balanced and all errors are equally costly.

Metric	Definition	When Misleading
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Imbalanced classes; asymmetric error costs
Precision	$TP/(TP+FP)$	When FN costs are high (ignores missed positives)
Recall	$TP/(TP+FN)$	When FP costs are high (ignores false alarms)
F1	Harmonic mean of P and R	When precision and recall have very different costs
ROC-AUC	Ranking quality across all thresholds	Highly imbalanced datasets (see PR-AUC instead)
PR-AUC	Precision-recall tradeoff across thresholds	When negative class is rare

6.2 The Confusion Matrix in Depth

For a binary problem, the confusion matrix is a 2×2 table. For K-class problems, it is a $K \times K$ matrix where entry (i,j) counts documents from class i classified as class j . Off-diagonal entries represent errors; the diagonal represents correct classifications.

Derived Metrics from Confusion Matrix

Precision = $TP/(TP+FP)$ **Recall** = $TP/(TP+FN)$ **Specificity** = $TN/(TN+FP)$

MCC (Matthews Correlation Coefficient) = $(TP \times TN - FP \times FN) / \sqrt{[(TP+FP)(TP+FN)(TN+FP)(TN+FN)]}$ is often the best single metric for binary imbalanced problems: it considers all four quadrants equally.

6.3 F-Measure Variants

The F_1 score weights precision and recall equally. The F_β score generalises this by allowing the practitioner to explicitly state the relative importance of recall vs. precision. This is a domain decision, not a technical one.

Scenario	Recommended Metric	Reasoning
Spam filter	$F\beta$ with $\beta=0.5$ (precision-heavy)	A false positive (blocking legitimate email) is worse than a false negative
Cancer screening	$F\beta$ with $\beta=2$ (recall-heavy)	A false negative (missed cancer) is far worse than a false positive
Fraud detection	PR-AUC	Very imbalanced; both precision and recall matter across thresholds
News categorisation	Weighted F1	Multiple classes; want overall performance across class sizes
Balanced binary task	F1 ($=F\beta$ with $\beta=1$)	Both errors cost equally; classes balanced

6.4 Macro vs. Micro vs. Weighted Averages — Worked Example

Suppose a 3-class classifier (A, B, C) produces these per-class results on a test set:

Class	Precision	Recall	F1	Support (N)
A (rare)	0.80	0.60	0.69	50
B (common)	0.90	0.95	0.92	400
C (medium)	0.75	0.80	0.77	150
Macro	0.82	0.78	0.79	600 (total)
Weighted	0.87	0.89	0.88	(by support)
Micro	0.88	0.88	0.88	(TP,FP,FN pooled)

Macro F1 = 0.79 treats the rare class A equally with the common class B, and is the right choice when class A performance matters as much as B (e.g. a rare but critical category). Weighted F1 = 0.88 reflects overall system performance weighted by class frequency — the right choice for reporting overall accuracy to stakeholders.

6.5 ROC-AUC vs. PR-AUC — When to Use Each

ROC-AUC (Area Under the Receiver Operating Characteristic Curve) measures the probability that a randomly chosen positive example is ranked above a randomly chosen negative example. It is threshold-independent and robust to class imbalance in the sense that it normalises both axes (TPR and FPR). However, ROC-AUC can be misleadingly optimistic when negatives vastly outnumber positives. A classifier with 1 positive per 1,000 negatives can achieve $\text{ROC-AUC} > 0.95$ while having precision of only 5% at its optimal threshold. PR-AUC focuses exclusively on the positive class (Precision on y-axis, Recall on x-axis) and does not reward good true-negative performance. It is the preferred metric for rare event detection.

Key Takeaway

Default metric selection rule: If positive class proportion $> 10\%$, ROC-AUC is appropriate. If positive class proportion $< 10\%$, use PR-AUC. Always report both for borderline cases.

6.6 Probability Calibration

Many classifiers produce scores that are not well-calibrated probabilities. A well-calibrated model predicts $P(y=1|x) = 0.7$ for a set of examples where roughly 70% are truly positive. Poorly calibrated models can be post-hoc calibrated using:

- Platt Scaling: Fits a logistic regression on the model's raw scores
- Isotonic Regression: Fits a non-parametric monotone function on the model's raw scores

Naive Bayes is severely overconfident (produces probabilities near 0 and 1 even for ambiguous examples). Logistic Regression is generally better calibrated but benefits from calibration when class prior adjustment is needed at test time.

7.1 K-Fold Variants and Their Trade-offs

Cross-validation addresses the fundamental problem of having limited data for both training and evaluation. By rotating which fold serves as the validation set, we obtain K estimates of generalisation performance and can report mean $\pm$ standard deviation — a far more honest characterisation than a single train/test split.

Variant	Description	Pros	Cons
Standard K-Fold	Random split into K equal folds	Simple; efficient	May not preserve class distribution
Stratified K-Fold	Each fold preserves class proportions	Essential for imbalanced data	Slight overhead
Group K-Fold	Keeps related examples in same fold	Prevents temporal/source leakage	Requires group labels
Time Series Split	Respects temporal ordering; no future data in train	Realistic for temporal tasks	Less data efficiency
Leave-One-Out (LOO)	$K=N$; each example is test set once	Unbiased for small N	Very slow; high variance

Critical: The Data Leakage Problem in NLP

Data leakage occurs when information from the test set influences the training process. In NLP pipelines, the most common leakage source is fitting the TF-IDF vectorizer on the full dataset before splitting. The correct approach: fit the vectorizer only on training folds, then transform validation folds. Use sklearn's Pipeline object to enforce this automatically — it re-fits all preprocessing steps at each fold.

7.2 Nested Cross-Validation for Unbiased Evaluation

Standard cross-validation with hyperparameter tuning produces an optimistically biased performance estimate: the validation set has been used to select hyperparameters, so it is no longer truly held-out. Nested cross-validation corrects this with two loops:

- Outer loop ($K=5$): Provides unbiased estimate of generalisation performance
- Inner loop ($K=3$ per outer fold): Selects optimal hyperparameters on training data only

Nested CV requires $K_{\text{outer}} \times K_{\text{inner}} \times n_{\text{hyperparameter_configs}}$ model fits. For $K=5 \times 3=15$ folds and 20 hyperparameter configurations, this means 300 model fits — computationally expensive but the gold standard for small datasets.

7.3 Hyperparameter Search Strategies

Strategy	Mechanism	Pros	Cons	When to Use
Grid Search	Exhaustive evaluation of all combinations	Guaranteed to find best in grid	Exponential in dimensions	< 4 hyperparameters
Random Search	Random sampling from distributions	60% fewer evaluations for same quality (Bergstra 2012)	May miss narrow optima	4–10 hyperparameters
Bayesian Optimisation	Models $P(\text{score} \text{params})$; exploits structure	Most efficient; handles complex surfaces	Complex to implement; serial	>10 parameters or expensive models
Successive Halving	Trains all configs briefly; eliminates worst half iteratively	Fast; scales to many configs	Poor for non-monotone performance curves	Large search spaces with fast models

8

Class Imbalance

Diagnosing and handling skewed label distributions

Class imbalance is ubiquitous in real-world text classification. Fraud: 0.1% positive rate. Rare disease mentions in clinical notes: 0.01%. Extremely toxic content in general social media: 0.5%. Relevant questions in a general-purpose QA system: 15%. In each case, a naive classifier learns to predict the majority class almost exclusively.

8.1 Diagnosing Class Imbalance

The imbalance ratio $IR = N_{\text{majority}} / N_{\text{minority}}$ is the primary diagnostic. General guidelines:

Imbalance Ratio	Severity	Recommended Strategy
1:1 to 4:1	Mild — often not problematic	None required; monitor weighted F1
4:1 to 10:1	Moderate — class weights recommended	<code>class_weight='balanced'</code> ; evaluate with F1 not accuracy
10:1 to 100:1	Severe — active intervention required	SMOTE or undersampling + class weights; PR-AUC as primary metric
> 100:1	Extreme — treat as anomaly detection	Anomaly detection methods; extreme oversampling; one-class classifiers

8.2 SMOTE and Its Variants

SMOTE (Synthetic Minority Over-sampling Technique, Chawla et al. 2002) generates synthetic minority class examples by interpolating between existing minority examples in the feature space. For each minority example, SMOTE selects k nearest minority neighbours and creates synthetic examples along the line segments connecting them.

SMOTE Synthetic Sample Generation

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \times (\mathbf{x}_{\text{nn}} - \mathbf{x}_i) \quad \text{where } \lambda \sim \text{Uniform}(0, 1)$$

x_i is a randomly selected minority example; x_{nn} is one of its k -nearest minority neighbours; λ is a random interpolation coefficient. The synthetic sample lies on the line segment between x_i and x_{nn} in feature space.

SMOTE Variant	Enhancement	Use When
SMOTE (original)	Interpolates between k -NN pairs	General imbalanced classification
Borderline-SMOTE	Only synthesises near the decision boundary	When boundary region is sparse
ADASYN	Synthesises more where density is low	Non-uniform minority distribution
SMOTE-Tomek	SMOTE + removes majority-minority boundary pairs	Cleaning noisy boundary

SMOTE Variant	Enhancement	Use When
SVM-SMOTE	Generates samples using SVM support vectors	When boundary is complex

Important limitation: SMOTE operates in the original feature space. For TF-IDF vectors, synthetic examples correspond to interpolations between word frequency profiles — these are not guaranteed to be linguistically coherent. For NLP tasks specifically, data augmentation through text-level operations (back-translation, synonym replacement) is often more effective than SMOTE.

8.3 NLP-Specific Data Augmentation for Minority Classes

Augmentation Method	Description	Quality	Cost
Synonym replacement	Replace k random non-stopwords with WordNet synonyms	Medium	Low
Back-translation	Translate to French/German, then back to English	High	Medium
Random insertion/deletion	Insert/delete random words	Low	Very low
Contextual word insertion (EDA)	Insert words predicted by masked LM (BERT)	High	Medium
Paraphrase generation	Generate paraphrases using T5 or GPT	Very high	High

Use Case: Rare Adverse Event Detection in Pharmacovigilance

A pharmaceutical company classifies social media posts for mentions of adverse drug reactions (ADRs). ADR mentions comprise 0.8% of posts. Strategy: (1) `class_weight={0:1, 1:125}`; (2) back-translation of all 240 ADR training examples (producing ~960 total); (3) F2 score as primary metric (recall-heavy, since missing an ADR report is worse than a false alarm). This combination improves recall from 0.41 to 0.78 at acceptable precision of 0.32.

Deep learning approaches to text classification have progressively supplanted classical methods on most benchmarks since 2014. This section surveys the main neural architectures, their inductive biases, and practical guidance for choosing among them.

9.1 Convolutional Neural Networks for Text (TextCNN)

Kim (2014) demonstrated that a single-layer CNN with multiple filter sizes achieves competitive performance on sentiment and question classification with far less training data than LSTM-based models. TextCNN applies 1D convolutional filters of width $\{2, 3, 4\}$ over the token embedding sequence, capturing local n-gram patterns at multiple scales. Architecture: Embed tokens $\rightarrow$ Apply parallel conv filters (100 filters each of width 2,3,4) $\rightarrow$ Max-over-time pooling (captures most activated pattern per filter) $\rightarrow$ Concatenate (300-dim) $\rightarrow$ Dropout $\rightarrow$ Dense $\rightarrow$ Softmax. Total parameters: $\sim 500K$ for typical settings, training in <5 minutes on CPU.

TextCNN Strength: Local Pattern Detection

TextCNN excels at sentiment classification because it can learn filters that activate on specific sentiment-bearing phrases ('not good', 'absolutely terrible', 'highly recommend') regardless of their position in the document. The max-pooling operation makes it translation-invariant — it finds the phrase regardless of whether it appears at the start or end of the review.

9.2 Bidirectional LSTM for Sequential Context

Long Short-Term Memory networks (Hochreiter & Schmidhuber, 1997) address the vanishing gradient problem of vanilla RNNs, allowing them to capture long-range dependencies. Bidirectional LSTMs (BiLSTM) process the sequence in both forward and backward directions, concatenating the hidden states to capture context from both directions. BiLSTMs outperform TextCNN on tasks requiring global sequential understanding: document-level sentiment, natural language inference, and tasks where the meaning of early tokens depends on later context ('The movie, despite its overhyped marketing, is actually great'). They require significantly more training time than TextCNN and are sensitive to sequence length.

9.3 BERT Fine-Tuning for Text Classification

BERT (Devlin et al., 2019) is pre-trained on 3.3 billion words of English text using two objectives: Masked Language Modelling (predict randomly masked tokens) and Next Sentence Prediction (predict if two sentences are consecutive). Fine-tuning adds a classification head (linear layer) on top of the [CLS] token representation and trains all parameters end-to-end on the target task.

BERT Classification Head

$$\text{logits} = W \times h_{[\text{CLS}]} + b \quad \text{where } h_{[\text{CLS}]} \in \mathbb{R}^{768} \text{ (BERT-base)}$$

$W \in \mathbb{R}^{(K \times 768)}$ and $b \in \mathbb{R}^K$ are the only newly initialised parameters. The remaining 110M BERT parameters are fine-tuned from pre-trained weights with a small learning rate ($1e-5$ to $5e-5$).

BERT Variant	Parameters	Speed	Best Use Case
BERT-base-uncased	110M	1x (baseline)	General classification; English text
DistilBERT	66M	1.6x faster	Production with latency constraints; 97% of BERT performance
RoBERTa-base	125M	1x	Higher accuracy than BERT; better pre-training
BERT-large	340M	0.4x slower	Maximum accuracy; offline/batch inference
DeBERTa-v3-base	183M	0.8x	State-of-the-art on most classification benchmarks
domain-specific BERT	110M	1x	Medical (BioBERT), legal (LegalBERT), finance (FinBERT)

9.4 Model Selection Guide

Constraint	Recommended Model	Rationale
< 500 training examples	SetFit or zero-shot NLI	Few-shot methods outperform fine-tuned BERT
500–5,000 examples	DistilBERT fine-tuned	Large pre-trained representation compensates for limited fine-tuning data
> 5,000 examples	RoBERTa or DeBERTa	Full fine-tuning yields maximum accuracy
Inference < 5ms	TF-IDF + LogReg or NB	Neural models are too slow without specialised hardware
Inference 5–50ms	DistilBERT with ONNX export	Optimised neural model within latency budget
Must explain decisions	L1 LogReg	Only model with intrinsically interpretable feature weights
Domain-specific (medical/legal)	Domain-specific BERT variant	Pre-training domain match is worth 3–8% F1

10.1 Zero-Shot Classification via Natural Language Inference

Zero-shot text classification (Yin et al., 2019) reformulates classification as a textual entailment task: does the premise document entail the hypothesis 'This text is about {label}'? A pre-trained NLI model (e.g. facebook/bart-large-mnli) scores the entailment probability for each candidate label, and the highest-scoring label is selected.

Zero-Shot NLI Classification

$$\text{score}(\mathbf{d}, \mathbf{l}) = \mathbf{P_NLI}(\text{entailment} \mid \text{premise}=\mathbf{d}, \text{hypothesis}=\text{'This text is about ' + l})$$

No task-specific training data required. The label space can be extended at inference time without any model retraining. Performance depends heavily on hypothesis template quality and is typically 10–20% below fully supervised fine-tuned models.

Zero-shot classification is most effective when label names are semantically transparent (e.g. 'sports', 'finance', 'technology') and least effective when labels are domain-specific jargon that the pre-trained model has not encountered (e.g. 'Tier-1 billing event', 'CAPA deviation').

10.2 Few-Shot Classification — SetFit

SetFit (Sentence Transformer Fine-Tuning, Tunstall et al. 2022) achieves GPT-3-level few-shot performance with 1,500× less compute. The method has two phases: (1) contrastive fine-tuning of a sentence transformer using in-batch negatives from the few-shot examples; (2) a logistic regression head trained on the fine-tuned embeddings.

Method	Examples Needed	Compute	Accuracy (SST-2)	API Access
GPT-3 few-shot	8–32 (in-context)	Very high (API)	~92%	Required
SetFit (8 examples)	8 per class	Low (local GPU)	~91%	Not required
Fine-tuned DistilBERT	1,000+	Medium	~92%	Not required
Multinomial NB (TF-IDF)	1,000+	Very low	~85%	Not required
Zero-shot NLI	0	Low	~78%	Not required

Research Spotlight: SetFit (Tunstall et al., 2022)

SetFit matches GPT-3 on 8-shot benchmarks while running entirely locally. The key insight is that contrastive sentence transformer fine-tuning — trained on positive (same class) and negative (different class) sentence pairs — rapidly adapts pre-trained sentence embeddings to the target label space, even with only 8 examples per class. The Hugging Face SetFit library provides a production-ready implementation in <10 lines of code.

10.3 Prompt Engineering for Classification

Large language models (LLMs) such as GPT-4, Claude, and LLaMA can perform text classification via prompting — the model is given instructions and examples in its context window, and outputs a label directly. This eliminates the need for any model training or fine-tuning. Effective prompt engineering for classification involves: (1) Clear label definitions in the system prompt; (2) Chain-of-thought instructions ('First explain your reasoning, then provide the label') for complex tasks; (3) A few representative examples (2–5) in the prompt; (4) Explicit output format constraints ('Respond with only one of: POSITIVE, NEGATIVE, NEUTRAL'). The primary limitations of prompt-based classification are: latency (500ms–2s per example vs. <10ms for a fine-tuned model), cost (API pricing at scale), and reproducibility (stochastic outputs require temperature=0 for deterministic results).

10.4 Open Research Problems

- Label efficiency: How can we achieve 90% of supervised performance with 1% of the labels?
- Compositional generalisation: Models fail on novel combinations of concepts seen separately in training
- Temporal robustness: How should classifiers handle concept drift without continuous re-labelling?
- Long-document classification: BERT's 512-token limit excludes full articles, legal documents, and research papers
- Multi-lingual transfer: A model trained in English performs poorly on structurally different languages
- Annotation efficiency: Active learning + LLM annotation is the emerging paradigm for cost-effective labelling

A.1 Complete sklearn Pipeline with Evaluation Dashboard

Full Classical Text Classification Pipeline

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import ComplementNB
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder

# 1. Build pipelines
lr_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1,2), max_features=50000,
                             min_df=2, sublinear_tf=True)),
    ('clf', LogisticRegression(C=1.0, solver='saga', penalty='l1',
                              class_weight='balanced', max_iter=1000))
])

nb_pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1,2), max_features=50000, min_df=2)),
    ('clf', ComplementNB(alpha=0.1))
])

# 2. Stratified 5-fold cross-validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
metrics = ['f1_weighted', 'f1_macro', 'roc_auc_ovr_weighted']
results = cross_validate(lr_pipeline, X_train, y_train,
                        cv=cv, scoring=metrics, return_train_score=True)

print(f'Weighted F1: {results["test_f1_weighted"].mean():.3f}
      +/- {results["test_f1_weighted"].std():.3f}')

# 3. Final evaluation on held-out test set
lr_pipeline.fit(X_train, y_train)
y_pred = lr_pipeline.predict(X_test)
print(classification_report(y_test, y_pred, digits=3))
```

A.2 Hyperparameter Tuning with RandomizedSearchCV

Randomised Hyperparameter Search

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform, randint

param_distributions = {
    'tfidf_max_features': [10000, 30000, 50000, 100000],
    'tfidf_ngram_range': [(1,1), (1,2), (1,3)],
    'tfidf_min_df': [1, 2, 3, 5],
```

```

'clf__C':          loguniform(0.001, 100),
'clf__penalty':    ['l1', 'l2'],
}

search = RandomizedSearchCV(
    lr_pipeline, param_distributions,
    n_iter=50,          # 50 random configurations
    cv=StratifiedKFold(5),
    scoring='f1_weighted',
    n_jobs=-1,         # use all CPU cores
    verbose=1,
    random_state=42
)
search.fit(X_train, y_train)
print('Best params:', search.best_params_)
print('Best CV F1:', search.best_score_)

```

A.3 BERT Fine-Tuning Skeleton (Hugging Face)

BERT Fine-Tuning for Sequence Classification

```

from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import TrainingArguments, Trainer
from datasets import Dataset
import evaluate

MODEL = 'distilbert-base-uncased' # swap for 'roberta-base' etc.
tokenizer = AutoTokenizer.from_pretrained(MODEL)

def tokenize(batch):
    return tokenizer(batch['text'], truncation=True, padding='max_length',
                      max_length=256)

# Convert to HuggingFace Dataset
train_ds = Dataset.from_dict({'text': X_train, 'label': y_train}).map(tokenize, batched=True)
val_ds = Dataset.from_dict({'text': X_val, 'label': y_val }).map(tokenize, batched=True)

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL, num_labels=num_classes)

args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    weight_decay=0.01,
    evaluation_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='f1',
)

f1_metric = evaluate.load('f1')
def compute_metrics(p):
    preds = p.predictions.argmax(-1)
    return f1_metric.compute(predictions=preds, references=p.label_ids, average='weighted')

```

```
trainer = Trainer(model=model, args=args,
                  train_dataset=train_ds, eval_dataset=val_ds,
                  compute_metrics=compute_metrics)
trainer.train()
```

A.4 SetFit Few-Shot Classification (8 examples per class)

SetFit with 8-shot learning

```
from setfit import SetFitModel, Trainer, TrainingArguments
from datasets import Dataset

# 8 examples per class (total: 8 * num_classes examples)
train_ds = Dataset.from_dict({'text': few_shot_texts, 'label': few_shot_labels})
val_ds = Dataset.from_dict({'text': X_val, 'label': y_val})

model = SetFitModel.from_pretrained(
    'sentence-transformers/all-mpnet-base-v2')

args = TrainingArguments(
    num_epochs=1,
    batch_size=16,
    num_iterations=20, # number of contrastive training pairs
)

trainer = Trainer(
    model=model, args=args,
    train_dataset=train_ds, eval_dataset=val_ds
)
trainer.train()
metrics = trainer.evaluate()
print(metrics) # typically 85-93% F1 with just 8 examples/class
```

Analytical Summary & Cross-Cutting Themes

Having traversed the full landscape of text classification — from foundational definitions through classical probabilistic models, feature engineering, rigorous evaluation, class imbalance handling, deep learning architectures, and cutting-edge few-shot methods — several cross-cutting themes emerge:

Theme	Detail	Practical Implication
Baseline-First Principle	NB and LR trained in seconds reveal data quality issues and establish achievable performance floors	Always run NB + LR before any neural model; measure the gap
Metric-Task Alignment	The evaluation metric must encode the actual cost of each error type in your domain	Define metrics before building models; involve domain stakeholders
Distribution Matters More Than Architecture	A perfect model on wrong-distribution training data will fail in production	Spend 60% of effort on data quality; 40% on modelling
Imbalance is Universal	Every real deployment has class imbalance; <code>class_weight='balanced'</code> should be the default	Never report accuracy as the primary metric on imbalanced tasks
Interpretability has Business Value	L1-LR's feature weights often have more production impact than 2% F1 from a neural model	Ask stakeholders whether they need to explain model decisions before choosing architecture
Few-Shot is Now Production-Ready	SetFit and LLM prompting have made high-quality classification viable with <20 labelled examples per class	Re-evaluate annotation budgets; 160 examples (20/class × 8) may suffice
Evaluation Rigour Separates Research from Engineering	Single train/test split results are insufficient; report K-fold mean ± std with explicit metric choice justification	Never report a result without confidence bounds
Monitoring is Not Optional	Text classifiers degrade silently due to concept drift; a deployed model with no monitoring is a ticking clock	Budget for monitoring before deployment, not after incidents

Recommended Study Progression

4. Implement Multinomial NB from scratch (without sklearn) on IMDb Reviews. Compare to sklearn's `ComplementNB`.
5. Train L1 and L2 Logistic Regression on AG News; visualise top-20 TF-IDF weights per class; explain three misclassified examples using the weight analysis.
6. Build a complete evaluation dashboard: per-class P/R/F1, macro/micro/weighted averages, confusion matrix, calibration curve.

7. Reproduce the class imbalance experiment: train on a 20:1 imbalanced dataset with and without `class_weight='balanced'`; compare PR-AUC curves.
8. Run 5-fold stratified CV inside a sklearn Pipeline to avoid data leakage; report mean F1 $\pm$ standard deviation.
9. Fine-tune DistilBERT on SST-2 for 3 epochs; compare to the LR baseline.
10. Run SetFit with 8 examples per class on SST-2; compare to the fine-tuned DistilBERT using only 8 examples.
11. Test zero-shot classification using facebook/bart-large-mnli on 20 Newsgroups without any training data.

The future of AI is bright, and you can be part of it!